

نوا

درس برنامه سازی وب · دانشگاه صنعتی شریف

گزارش نهایی پروژه

فاز اول (فرانت اند) و فاز دوم (بک اند و ادغام)

محمد · عماد · مهدی

۱. معرفی پروژه

نوا یک سرویس استریم موسیقی است با چهار نقش کاربری — شنونده، هنرمند، پشتیبان و مدیر سامانه — و سه سطح اشتراک (پایه، نقره‌ای، طلایی). پروژه در دو فاز ساخته شد: فاز اول رابط کاربری کامل با داده‌ی ماک، و فاز دوم بک‌اند واقعی و ادغام آن با فرانت‌اند.

وضعیت فعلی پروژه

بخش	فناوری	حجم
فرانت‌اند	Next.js 16 · React 19 · TypeScript · Tailwind v4 · Zustand	۱۳۱ فایل · ۷۴۰ تست
بک‌اند	Django 5.2 · Django REST Framework · JWT · PostgreSQL	۸۷ فایل · ۱۸۸۰ تست
API	۱۸ مسیر REST با خروجی camelCase	—

اجرای کامل پروژه با یک دستور: `docker compose up --build` — راهنمای کامل اجرا در فایل `README.md` ریشه‌ی مخزن آمده است.

۲. نقش و کارهای هر عضو

کار روی سه شاخه‌ی مستقل Git پیش رفت و از طریق Pull Request ادغام شد.

فاز اول — فرانت‌اند

عضو	کارها	شاخه
عماد	زیرساخت و ساختار پروژه، ورود و ثبت‌نام، صفحه خانه، نمایه کاربر و مدیریت آن، تنظیمات، اعلانات، و PWA	feature/core-auth
مهدی	پخش‌کننده موسیقی (صف، تکرار، شافل، متن آهنگ)، آرشیو آلبوم و تک‌آهنگ، پلی‌لیست‌ها، نمایه هنرمند	feature/app-player
محمد	استودیوی هنرمند، داشبورد پشتیبان و مدیر، تیکت‌ها، احراز هویت هنرمندان، حسابرسی، و تست‌ها	feature/dashboard-studio

فاز دوم — بک‌اند و ادغام

در این فاز کار بیشتر یکپارچه بود تا تفکیک‌شده، چون بک‌اند یک واحد به‌هم‌پیوسته است:

- **محمد:** طراحی مدل‌ها و معماری اپ‌ها، احراز هویت و سطوح دسترسی، گزارش‌گیری و حسابرسی، اشتراک و درگاه پرداخت، تست‌ها، و داکرایز کردن.
- **عماد:** بازبینی ساختار پروژه و قراردادهای کد، بخش حساب کاربری و همگام‌سازی تنظیمات.
- **مهدی:** بخش کاتالوگ و پلی‌لیست، آپلود رسانه، و اتصال پخش‌کننده به ثبت استریم در سرور.
- **مشترک:** ادغام فرانت‌اند با API، دوزبانه‌سازی رابط کاربری، بازبینی کد یکدیگر، و تهیه‌ی همین گزارش.

۳. قوانین توسعه و فرآیند کار

نام‌گذاری

مورد	قرارداد
کامپوننت‌های React	PascalCase — مثل <code>WorkItem</code> ، <code>PlayerBar</code>
فایل‌ها و پوشه‌ها	kebab-case — مثل <code>publish-work-dialog.tsx</code>
توابع و متغیرها (TS)	camelCase — مثل <code>toggleShuffle</code>
استورها	با پیشوند <code>use</code> — مثل <code>usePlayer</code> ، <code>useDb</code>
کلاس‌ها و ماژول‌های پایتون	PascalCase برای کلاس، <code>snake_case</code> برای تابع و فیلد
اپ‌های جنگو	اسم جمع و حوزه‌محور — <code>playlists</code> ، <code>catalog</code> ، <code>accounts</code>
شاخه‌های Git	<code>feature/<area></code> — مثل <code>feature/dashboard-studio</code>
پیام کامیت	feat/fix/test/docs/chore — Conventional Commits

فرآیند

- هر عضو روی شاخه‌ی خودش کامیت‌های کوچک و معنادار می‌زد.
- ادغام کد فقط از طریق Pull Request و پس از بازبینی انجام می‌شد.
- هیچ کدی بدون سبز بودن تست‌ها ادغام نشد.
- مقادیر محرمانه و وابسته به محیط در فایل `env` نگهداری می‌شوند و در گیت قرار نمی‌گیرند؛ به جای `env.example` بدون مقدار واقعی هست.

۴. ساختار دو پروژه و توجیه آن

فرانت‌اند

```
src/
├─ app/           (App Router) صفحه‌ها و مسیرها
│  ├─ (auth)/     ورود، ثبت‌نام، فراموشی رمز
│  ├─ (app)/      تجربه‌ی شنونده و هنرمند + پخش‌کننده
│  └─ dashboard/  پنل پشتیبان و مدیر (بدون پخش‌کننده)
├─ components/   اجزای رابط کاربری، تفکیک‌شده بر اساس حوزه
└─ lib/          منطق و داده
```

توجیه: مسیرها با «گروه مسیر» به سه تجربه‌ی مستقل تقسیم شده‌اند، چون هر گروه چیدمان و سطح دسترسی متفاوتی دارد. مهم‌تر اینکه منطق در lib از ظاهر در components جدا است؛ همین جداسازی باعث شد در فاز دوم بتوانیم لایه‌ی داده را عوض کنیم بدون آنکه کامپوننت‌ها تغییر کنند.

بک‌اند

```
backend/
├─ nava/          تنظیمات و آدرس‌دهی پروژه
├─ accounts/      کاربر، ترجیحات، هنرمند، دنبال‌کردن
├─ catalog/       آهنگ، آلبوم، اعتبار هنرمندان
├─ playlists/     پلی‌لیست و آیتم‌های آن
├─ engagement/    اعلان و تیکت پشتیبانی
├─ subscriptions/ قیمت‌ها، اشتراک، پرداخت، درگاه
├─ reports/       لاگ استریم، گزارش‌ها، تسویه هنرمندان
└─ common/        مدل و کلاس‌های پایه، دسترسی‌ها، ثابت‌ها
```

توجیه: به جای یک اپ بزرگ، هفت اپ کوچک و حوزه‌محور ساختیم. هر اپ یک مسئولیت دارد و مستقل قابل فهم است؛ مثلاً همه‌ی منطق پرداخت داخل subscriptions است و بقیه‌ی سیستم فقط از طریق سرویس‌های آن با پرداخت کار می‌کند. اپ common هم چیزهای مشترک را نگه می‌دارد تا تکرار نشوند.

۵. مدل‌های بک‌اند، روابط و توجیه وجودشان

مدل	رابطه‌ی کلیدی	چرا وجود دارد
User	—	حساب کاربری با نقش و اشتراک؛ ایمیل شناسه‌ی ورود است
UserPreferences	OneToOne → User	تنظیمات کاربر (از جمله زبان) روی سرور تا بین همه‌ی دستگاه‌ها یکسان بماند
Artist	OneToOne → User	پروفایل هنرمند جداست چون کاتالوگ به آن ارجاع می‌دهد و فرآیند تایید هویت اینجاست
Follow	User × Artist	جدول واسط دنبال کردن؛ مبنای شمارش دنبال کننده و ارسال اعلان اثر جدید
Album / Song	Song → Album (FK)	کاتالوگ؛ تک‌آهنگ آلبوم ندارد و همین یک فیلد هر دو حالت را پوشش می‌دهد
SongArtist	through (M2M)	ترتیب اعتبار هنرمندان را نگه می‌دارد تا هنرمند اصلی از همکار تفکیک شود
AlbumArtist	through (M2M)	همان منطق برای آلبوم
Playlist	FK → User	پلی لیست کاربر؛ سقف تعداد بر اساس اشتراک اعمال می‌شود
PlaylistItem	through (M2M)	ترتیب آهنگ‌ها در پلی لیست را حفظ می‌کند
Notification	FK → User	اعلان‌ها فقط سمت سرور ساخته می‌شوند (انتشار اثر، نتیجه تایید، تیکت)
Ticket / TicketMessage	FK زنجیره‌ای	تیکت پشتیبانی و رشته‌ی پیام‌های آن
PlatformSettings	تک‌ردیفی	قیمت‌ها و نرخ پاداش؛ مدیر بدون تغییر کد آن‌ها را عوض می‌کند
Subscription	FK → User	دوره‌ی خریداری شده با تاریخ شروع و پایان؛ مبنای انقضا
Payment	FK → User	هر تراکنش با وضعیت در حال انجام / موفق / ناموفق پیگیری می‌شود
StreamEvent	User × Song	لاگ هر پخش — ستون فقرات آمار (توضیح پایین)
Payout	FK → Artist	تسویه ماهانه هنرمند، محاسبه شده از همان لاگ

چرا StreamEvent مهم‌ترین تصمیم مدل‌سازی ماست

یک جدول لاگ ساده، هم‌زمان سه مسئله را حل کرد:

۱) سقف استریم روزانه: کافیت رکوردهای امروز شمرده شوند — پس هیچ کرون‌جاب و ریست نیمه‌شب‌ی لازم نیست.

۲) آمار آهنگ‌ها و هنرمندان.

۳) گزارش مالی ماهانه که به «شنونده‌ی منحصربه‌فرد» نیاز دارد و بدون تاریخچه اصلاً قابل محاسبه نیست.

۶. توجیه نگهداری‌پذیری کد

۱) تنها منبع حقیقت

قوانین اشتراک (سقف استریم، سقف پلی‌لیست، دسترسی زودهنگام) در یک فایل ثابت تعریف شده‌اند و همه‌ی بخش‌ها از همان می‌خوانند. تغییر یک محدودیت یعنی ویرایش یک خط، نه جست‌وجو در کل پروژه.

۲) داده‌ی پویا به جای عدد ثابت در کد

قیمت اشتراک‌ها و نرخ پاداش هنرمندان در پایگاه داده نگهداری می‌شوند نه در کد. مدیران‌ها را از داشبورد عوض می‌کند و کل سامانه بلافاصله تبعیت می‌کند — دقیقاً همان چیزی که صورت پروژه خواسته بود.

۳) درز فاز دوم

از فاز اول تمام تغییرات داده در یک لایه‌ی متمرکز جمع شده بود و شکل داده‌ها عمداً نزدیک به خروجی بک‌اند طراحی شده بود. نتیجه این شد که در فاز دوم فقط بدنه‌ی همان توابع به فراخوانی API تبدیل شد و ده‌ها کامپوننتی که از آن‌ها استفاده می‌کنند اصلاً تغییر نکردند.

۴) جداسازی سیاست از مکانیزم

پخش‌کننده فقط می‌داند «چطور پخش کند» و هیچ اطلاعی از اشتراک و دیتابیس ندارد؛ بررسی مجوز پخش در لایه‌ی جداگانه است. همین باعث شد استور پخش‌کننده کاملاً مستقل و قابل تست بماند.

۵) الگوهای طراحی

- **الگوی استراتژی**: درگاه پرداخت یک رابط مشترک با دو پیاده‌سازی دارد (زرین‌پال و آزمایشی آفلاین). تعویض درگاه فقط تغییر یک تنظیم است، نه تغییر کد.
- **کلاس پایه‌ی مشترک**: شناسه‌ی خوانا و تاریخ ساخت یک‌بار در مدل پایه تعریف شده و همه‌ی مدل‌ها از آن ارث می‌برند.
- **لایه‌ی سرویس**: منطق پیچیده (پرداخت، گزارش‌گیری، پیشنهاددهنده) در فایل‌های services جدا از ویوها قرار دارد.
- **پرهیز از کد تکراری**: با ViewSet و Serializer های DRF، به جای نوشتن دستی پنج ویو برای هر منبع، یک کلاس نوشته می‌شود.

۶) تست به عنوان تور ایمنی

۱۸۸ تست بک اند و ۷۴ تست فرانت اند. تست ها دو لایه اند: تست های واحد که توابع خالص را مستقیم صدا می زنند (بدون دیتابیس، در کسری از ثانیه)، و تست های یکپارچگی که مسیر کامل درخواست را می سنجند. نتیجه این است که وقتی تستی می شکند، معمولاً دقیقاً می دانیم کدام تابع مقصر است. هیچ تستی به شبکه وابسته نیست: درگاه پرداخت در تست ها روی حالت آفلاین قفل شده و پاسخ های زرین پال با نمونه های واقعی ثبت شده شبیه سازی می شوند.

۷. نقش هوش مصنوعی در توسعه

از هوش مصنوعی به عنوان یک هم تیمی کمکی استفاده کردیم، نه به عنوان جایگزین تصمیم گیری. تصمیم های معماری — تقسیم اپ ها، طراحی مدل ها، انتخاب «درز فاز دوم» — با خودمان بود و هوش مصنوعی بیشتر در این کارها کمک کرد:

- تولید سریع کدهای تکراری مثل serializer ها، فرم ها و جدول ها.

- نوشتن تست برای حالت های مرزی که ممکن بود از قلم بیفتند.

- بازبینی کد و پیدا کردن ناسازگاری بین بخش ها.

- مستندسازی و توضیح بخش های پیچیده.

در مقابل، هر خروجی را خودمان بازبینی و اجرا کردیم؛ چند مورد از پیشنهادهای اولیه ی آن را رد یا اصلاح کردیم که در بخش ۹ به آن اشاره شده است.

۸. نمونه کد تولید شده با کمک هوش مصنوعی

نمونه ی فاز اول — منطق پخش تصادفی (شافل)

این تابع در استور پخش کننده است. نکته ی ظریفش این است که آهنگ در حال پخش باید سرِ صف بماند و با خاموش شدن شافل، ترتیب اصلی دقیقاً برگردد؛ برای همین دو صف جداگانه نگه داشته می شود.

frontend/src/lib/stores/player-store.ts

```
toggleShuffle: () => {
  const { shuffle, queue, baseQueue, index } = get();
  const playing = queue[index] ?? null;
  if (!shuffle) {
    // Turn on: keep current track first, shuffle the rest.
    const rest = shuffled(baseQueue.filter((id) => id !== playing));
    set({ shuffle: true,
          queue: playing ? [playing, ...rest] : shuffled(baseQueue),
          index: 0 });
  } else {
    // Turn off: restore the canonical order around the current track.
    set({ shuffle: false, queue: baseQueue,
          index: playing ? Math.max(0, baseQueue.indexOf(playing)) : 0 });
  }
},
```

نمونه‌ی فاز دوم — انقضای اشتراک

این بخش پس از یک بازبینی اضافه شد: متوجه شدیم اشتراک منقضی‌شده همچنان امتیازات خود را نگه می‌داشت. راه‌حل این بود که همه‌ی بررسی‌های دسترسی به‌جای فیلد ذخیره‌شده، «سطح مؤثر» را بخوانند تا انقضا بدون نیاز به هیچ زمان‌بندی، بلافاصله اعمال شود.

backend/accounts/models.py

```
@property
def subscription_is_active(self) -> bool:
    """A paid plan is only in force until its renewal date passes."""
    if self.subscription_tier == SubscriptionTier.BASIC:
        return True
    return (self.subscription_renews_at is not None
            and self.subscription_renews_at > timezone.now())

@property
def current_tier(self) -> str:
    """A lapsed paid plan falls back to basic."""
    return (self.subscription_tier if self.subscription_is_active
            else SubscriptionTier.BASIC)
```

نمونه‌ی دوزبانه‌سازی — ترجمه با جای‌گذاری مقدار

متن‌های برنامه به فارسی نوشته شده‌اند و خود همان متن فارسی «کلید» ترجمه است؛ اگر ترجمه‌ای نبود، متن فارسی نمایش داده می‌شود نه یک کلید خام. نکته‌ی مهم، حفظ جای‌گذارها است تا جمله‌هایی که ترتیب کلماتشان بین دو زبان فرق دارد درست بمانند.


```
/**
 * Translate a Persian source string, falling back to it when untranslated.
 *
 * `translate("{n} آهنگ", "en", { n: 5 })` → `"5 tracks"`
 */
export function translate(text: string, language: Language, vars?: Vars): string {
  return interpolate(language === "fa" ? text : EN[text] ?? text, vars);
}
```

۹. نقاط قوت و ضعف هوش مصنوعی

در توسعه‌ی فرانت‌اند

مشاهده‌ی ما	
ساخت سریع کامپوننت‌های تکراری (کارت، جدول، فرم) و رعایت یکدست الگوهای موجود؛ کمک خوب در نوشتن استایل و چیدمان RTL	قوت
تبدیل توضیح صورت پروژه به یک UI اولیه‌ی قابل کار در زمان کوتاه	قوت
گاهی از API‌های قدیمی نسخه‌های پیشین Next.js و کتابخانه‌ها استفاده می‌کرد و باید با مستندات نسخه‌ی واقعی اصلاح می‌شد	ضعف
در جزئیات ظریف تجربه‌ی کاربری (مثل رفتار دکمه‌ی «قبلی» یا حفظ ترتیب پس از شافل) پیشنهاد اولیه‌اش ساده‌انگارانه بود و خودمان اصلاحش کردیم	ضعف
مدیریت خطا را با catch خالی می‌نوشت. نتیجه این بود که هر خطایی — حتی خاموش بودن بک‌اند — به کاربر «این ایمیل قبلاً ثبت شده» نشان می‌داد و مدتی فکر می‌کردیم ثبت‌نام خراب است، در حالی‌که API سالم بود	ضعف

مشاهده‌ی ما	
بسیار خوب در کدهای قالبی ViewSet، serializer، DRF، کلاس‌های دسترسی — و در نوشتن تست برای حالت‌های مرزی	قوت
کمک مؤثر در بهینه‌سازی کوئری‌ها و جلوگیری از مشکل N+1	قوت
تمایل به تولید تست‌های یکپارچگی به جای تست واحد؛ ابتدا تقریباً همه‌ی تست‌ها از لایه‌ی HTTP رد می‌شدند و بعداً تست‌های واحد را اضافه کردیم	ضعف
الزامات ضمنی صورت پروژه را از قلم می‌انداخت؛ انقضای اشتراک و فرمول واقعی پاداش تنها پس از بازبینی دستی خودمان با متن پروژه پیدا و اضافه شدند	ضعف
سرویس‌ورکر PWA که تولید کرده بود «هر پاسخ موفق» را کش می‌کرد؛ یعنی پاسخ /api/auth/me/ شامل نام و ایمیل کاربر روی دستگاه باقی می‌ماند و پس از خروج از حساب هم قابل خواندن بود. این را خودمان در تست PWA پیدا کردیم و کش را به «فقط منابع هم‌ریشه» محدود کردیم	ضعف

جمع‌بندی: هوش مصنوعی در «چگونه نوشتن» بسیار کمک کرد، ولی «چه چیزی باید نوشته شود» همچنان نیازمند خواندن دقیق صورت پروژه و قضاوت خودمان بود. الگوی مشترک هر سه ضعف جدی‌ای که پیدا کردیم یکی بود: کد در حالت خوش‌بینانه درست کار می‌کرد و در حالت خطا یا مرزی اشتباه رفتار می‌کرد — و این‌ها فقط با تست کردن دستی خودِ محصول پیدا شدند، نه با خواندن کد.

۱۰. وضعیت پیاده‌سازی

برای شفافیت، وضعیت موارد خواسته‌شده را این‌جا می‌آوریم:

مورد	وضعیت	توضیح
مدل‌ها، REST، اندپوینت‌های CRUD	انجام شد	۱۸ مسیر
ارسال و دریافت فایل	انجام شد	آهنگ، کاور، آواتار
ادغام با فرانت‌اند	انجام شد	هر چهار نقش
سطوح دسترسی (نقش و اشتراک)	انجام شد	سمت سرور
گزارش‌گیری سمت سرور	انجام شد	بدون محاسبه در فرانت
اشتراک و انقضا و تمدید	انجام شد	سطح مؤثر، بدون زمان‌بندی
درگاه پرداخت	انجام شد	سندباکس زرین‌پال + درگاه آزمایشی
تست بک‌اند (حداقل ۱۵)	۱۸۸ تست	واحد + یکپارچگی
تغییر زبان رابط کاربری	انجام شد	فارسی/انگلیسی با چیدمان RTL/LTR
داکرایز کردن (امتیازی)	انجام شد	docker compose up
پیشنهاددهنده آهنگ (امتیازی)	انجام شد	مبتنی بر تاریخچه‌ی پخش
PWA (امتیازی)	انجام شد	نصب‌شدنی و کارکرد آفلاین

دوزبانه‌سازی — آنچه در بازبینی نهایی تکمیل شد

در نسخه‌ی پیشین این گزارش، تغییر زبان «ناقص» علامت خورده بود: گزینه‌اش وجود داشت ولی متن‌ها ترجمه نشده بودند. این بخش اکنون کامل شده است:

- واژه‌نامه‌ای با ۴۳۶ عبارت که کل رابط کاربری را پوشش می‌دهد.
- چیدمان با تغییر زبان از راست‌به‌چپ به چپ‌به‌راست برمی‌گردد؛ منوها هم از سمت درست باز می‌شوند.
- اعداد، تاریخ و واحد پول تابع زبان‌اند: ارقام فارسی و تقویم جلالی در فارسی، ارقام لاتین و تقویم میلادی در انگلیسی.
- قلم هم عوض می‌شود: استعداد برای فارسی و یک قلم لاتین برای انگلیسی.

یک تصمیم آگاهانه

نام آهنگ‌ها، آلبوم‌ها و هنرمندان **ترجمه نمی‌شوند**. این‌ها داده‌ی کاتالوگ‌اند نه متن رابط کاربری — همان‌طور که اسپاتیفای هم نام آهنگ‌ها را ترجمه نمی‌کند. بنابراین در حالت انگلیسی، رابط کاربری کاملاً انگلیسی است ولی نام آثار فارسی می‌ماند؛ این نقص نیست، رفتار درست است.

درگاه پرداخت

پیاده‌سازی روی **سندباکس زرین‌پال** است که به حساب پذیرنده نیاز ندارد. مسیر کامل است: دریافت authority، هدایت کاربر به صفحه‌ی پرداخت، و سپس تأیید سمت سرور. هیچ اشتراکی تا وقتی تأیید موفق نشود فعال نمی‌شود — بازگشت مرورگر به آدرس callback به‌تنهایی هیچ چیز را اثبات نمی‌کند و authority جعلی رد می‌شود. اگر سندباکس در دسترس نبود، با یک تنظیم (PAYMENT_GATEWAY=fake) همان مسیر به‌صورت آفلاین اجرا می‌شود.

۱۱. اجرای پروژه

راهنمای کامل در README.md ریشه‌ی مخزن است. خلاصه‌اش:

```
docker compose up --build
# frontend    → http://localhost:3000
# backend API → http://localhost:8000/api
# پنل مدیریت → http://localhost:8000/admin
```

داده‌های نمایشی خودکار ساخته می‌شوند. رمز عبور همه‌ی حساب‌ها: nava1234

ایمیل	نقش	چه چیزی را نشان می‌دهد
gold@nava.app	شنونده‌ی طلایی	دسترسی زود هنگام، آمار، پلی لیست نامحدود
silver@nava.app	شنونده‌ی نقره‌ای	اشتراک منقضی‌شده و نیاز به تمدید
basic@nava.app	شنونده‌ی پایه	سقف ۶۰ استریم روزانه و ۶ پلی لیست
artist@nava.app	هنرمند تایید شده	استودیو، انتشار اثر، آمار و درآمد
pending@nava.app	هنرمند در انتظار	حالت «در انتظار تایید» و مسدود بودن انتشار
support@nava.app	پشتیبان	تیکت‌ها و احراز هویت هنرمندان
admin@nava.app	مدیر سامانه	حسابرسی، کنترل قیمت‌ها و گزارش درآمد

همین فهرست پایین صفحه‌ی ورود هم هست و با یک کلیک وارد می‌شوید. سرویس‌ورکر PWA فقط در بیلد تولیدی فعال است (npm run build && npm start)، نه در حالت توسعه.